

Uganda NDC Data Explorer

Complete application and engineering guide

Revision: 25 September 2026

Repository: `ndc-data-explorer-e051f914`

Audience: engineers, maintainers, analysts, and implementation partners

Purpose. This is the single long-form guide to the current application: what users do, how each workflow is built, where its evidence comes from, how Climate TRACE is queried, and how to operate and extend the system. It describes the code as it stands in this repository. Data release dates are a checked snapshot; verify them again before citing a newer release.

Contents

1. Application at a glance
2. From entry to a decision
3. Runtime architecture
4. Climate TRACE: the implementation contract
5. Targets, indicators, and derived outputs
6. Documents and AI workflow
7. Protected writes and persistence
8. Local development and production operation
9. Tests, release checks, and known limits
10. Maintenance map and external references
11. API quick map and troubleshooting
12. Qlik handoff extract

1. Application at a glance

The Uganda NDC Data Explorer is a web decision workspace for examining Uganda's Updated Nationally Determined Contribution (NDC), observed emissions, policy evidence, mitigation options, delivery data, and indicative forecasts. Its central question is whether available observations and planned actions are consistent with national climate commitments. It is a planning and evidence-navigation tool, not Uganda's official greenhouse-gas inventory or a certification system for projects.

A user enters the site, chooses Uganda, and can move between the emissions map, District Translator, the NDC dashboard, sector and scenario tools, policy documents, finance screening, forecasts, and delivery workflows. The top bar has shortcuts; **All tools** groups the longer menu under Explore, Plan & deliver, and Manage & learn. A role selector changes workspace visibility. At present, no site-wide login is required. Role choice is a browser preference; it never grants permission to write protected data. The separate operator unlock still protects ingestion and other server-side writes.

The application deliberately keeps four kinds of information distinct:

1. **Official intent:** Uganda's 2022 NDC targets, business-as-usual (BAU) references, sector activities, and national policy commitments bundled with the application.
2. **Observed emissions:** live-derived Climate TRACE estimates, fetched through the backend. Aggregate emissions power dashboard totals; located source records power maps and the District Translator.
3. **Supporting evidence and screening:** Climate Policy Radar (CPR) document exports and passages, multilateral climate fund (MCF) project records, KCI policy case analogies, and indicative finance and forecast models.
4. **User or ministry inputs:** browser-local activity drafts and, when Postgres and an operator session are configured, mapped ingestion observations.

The most important reading rule is that a target, a modeled forecast, a located-source subtotal, and an observed national aggregate are different measurements. Their labels, provenance, and coverage should travel with any screenshot or export.

2. From entry to a decision

2.1 Entry and navigation

Open `/` or `/select-country`, choose Uganda, and select a role if useful. The `/auth` route currently redirects to country selection because `LOGIN_AUTH_ENABLED` is false in `frontend/src/lib/auth-config.ts`. The Supabase authentication implementation remains in the codebase for later reactivation; the current browser identity is local and stable within the application. Open **All tools** to find the full route collection. There is no persistent left sidebar.

A useful first walkthrough is: open `/map` to see where located sources appear; open `/dashboard` to compare a national sector with its NDC ceiling; switch geography to inspect one of 56 mapped GADM districts; open `/district-translator` to select a 2020 administrative boundary and inspect source-centroid coverage; then inspect `/documents`, `/policy-impact`, and `/climate-finance` to connect evidence and possible interventions. Before briefing someone, verify `/api/health` OR `/api/v1/health/full` and check whether `USE MOCK DATA` is false.

2.2 Dashboard workflow

The `/dashboard` cockpit is the core observed-versus-intended workspace. Its columns hold targets and activities, observed emissions and source traceability, and progress or evidence context. Choose a sector and year, then inspect its official 2030 BAU and NDC ceiling alongside Climate TRACE history. Select a bar to view its value, methodology, upstream path, and source link. The source panel and top-emitter panel are drill-downs; their listed points are not expected to sum to the headline aggregate.

Switch between **National** and **District**. National history begins in 2015; district series begin in 2021. The district selector exposes 56 Climate TRACE/GADM areas. National NDC targets are not allocated to districts, so the district screen provides emissions context without an on-track/off-track district score. Selecting a target or URL sector must preserve the selected target in app state; the dashboard columns share the sector, geography, target, and time mode.

The NDC 2030 target is BAU-relative. In a growing sector, the target ceiling can be above its 2015 observed baseline while still being below the 2030 no-policy trajectory. The progress display therefore evaluates proximity to the target ceiling, rather than a simplistic reduction since 2015. Climate TRACE model estimates and Uganda's official inventory may also differ in scope and method; the UI warns about material divergence instead of implying exact equivalence.

2.3 Map and District Translator

`/map` renders located Climate TRACE sources on a MapLibre satellite map. The map feed can contain up to 3,000 upstream rows, omits unlocated emissions, and reports whether it was truncated. It is a spatial visualization, not a national emissions total. Source records may represent facilities or administrative aggregations; their spatial precision differs.

`/district-translator` currently starts with district selection. Choose a district by name or click its boundary, select a year and sector filters, review coverage and trend, and export CSV or GeoJSON. The Draw control, point controls, and drawing instructions are temporarily hidden by `DRAW_ENABLED=false` in `frontend/src/pages/DistrictTranslator.tsx`. The map handlers and server polygon analysis code remain. The translator's 135 pinned 2020 UBOS district boundaries are **not** the 56 GADM districts used by the dashboard. Their IDs cannot be treated as interchangeable. The translator includes a complete source value when the source centroid falls in the chosen shape; this is not a measured territorial allocation.

2.4 Other decision routes

- `/sector-classification` helps users connect reporting codes and sector definitions to the app's NDC categories.

- `/scenario-analysis` compares actions, timing, and policy evidence; interpret scenario outputs as assumptions, not observations.
- `/ai-2030` shows indicative sector trends and uncertainty against targets. The observed history and forward model must be labeled separately.
- `/policy-impact` turns a selected intervention into a transparent socio-economic forecast based on KCI case analogies and the Transition Element Framework (TEF). It presents matched cases and trade-offs.
- `/climate-finance` screens indicative abatement costs and fund pathways and links relevant MCF evidence. These values are not audited investment estimates.
- `/documents` searches a built CPR metadata corpus, searchable CPR passages, MCF projects, and an illustrative intervention pathway. `/documents/view` offers a document preview and AI-assisted PDF analysis when a server-side OpenAI key is configured.
- `/ingest` starts with Quick scan and offers a mapped Data Pipeline import. Protected writes require the separate operator unlock; successful mapped observations can contribute dashboard provenance when Postgres is configured.
- `/mwp-marketplace` presents pre-authored mitigation deals and scorecards. `/my-work` holds browser-local activity drafts and submissions. Advanced and legacy routes include `/library`, `/risk`, `/executive`, `/delivery`, `/indicators`, and related cockpit slices.

The in-app `/docs` page contains a user guide and system design view. Its copy is built from `frontend/src/data/user-guide-content.ts` and bundled `docs/dev/system-design.md`.

3. Runtime architecture

3.1 Request path

The frontend is a React 18, TypeScript, Vite single-page application. React Router chooses pages; TanStack Query and the emissions context manage remote data. In local development Vite listens on port 8080 and proxies `/api` to Express on port 8787. The browser normally uses same-origin `/api/v1/...` requests through `frontend/src/lib/api.ts`; it does not contain a Climate TRACE key or query Climate TRACE directly.

Express is assembled in `backend/server/createApp.js`, launched locally by `backend/server.js`, and run in production through `api/index.js`. Route modules validate inputs and call service modules. `config/` holds target and upstream request settings; `shared/` holds units and schemas. In production the Vite build is copied to `public/`, while Vercel rewrites `/api/*` to the serverless API and other paths to the SPA. `vercel.json` defines the build and rewrites.

Simplified data flow

```
Browser page -> frontend API client -> /api/v1 route -> backend service
                                     |-> Climate TRACE v7
                                     |-> bundled policy and target data
                                     |-> Postgres (when configured)
                                     |-> AI provider (where enabled)
route response -> React Query / page state -> labeled chart, panel, or export
```

The shared browser state includes `CountryContext`, `CurrentRoleProvider`, `OperatorSessionProvider`, `dashboard useAppState`, and `EmissionsDataProvider`. The role provider controls presentation; the operator provider checks a server-issued cookie. These should not be conflated in authorization decisions.

3.2 Code map

- `frontend/src/App.tsx`, `frontend/src/pages/`, and `frontend/src/lib/navigation.ts`: route mounting, screens, and All tools navigation.
- `frontend/src/components/columns/`, `frontend/src/context/EmissionsDataContext.tsx`, and `frontend/src/lib/dashboard-ai-*`: dashboard layout, remote data, and cited AI context.
- `frontend/src/components/map/`, `frontend/src/pages/DistrictTranslator.tsx`: visual source map, pinned-boundary selection, and dormant drawing UI.
- `backend/routes/emissions.js`: public emissions, map, translator, spatial confidence, predictions, and related routes.
- `backend/services/climatetrace.js`, `climateTraceTimeseries.js`, `emissionsData.js`, and `backend/services/translator/`: upstream adapters, caches, strict aggregation, and polygon reports.
- `config/climateTrace.js`, `config/ugandaDistrictGadm.js`, `config/ndcTargets.js`, `shared/emissionsUnits.js`, and `shared/schemas/`: API construction, geography mapping, sector definitions, units, and validation.
- `backend/routes/documents.js`, `policyAi.js`, `dashboardAi.js`, `policyImpact.js`, `ingest.js`, and `authSession.js`: document search, AI assistance, policy forecast, imports, and operator sessions.
- `data/policy/`, `data/policy-cases/`, `database/`, and `scripts/`: built evidence corpora, policy analogies, persistence, build and verification jobs.

Services should own upstream interpretation; routes should validate and shape API responses; pages should display the response's provenance and unavailable states. Recompute neither emission units nor source coverage ad hoc in a visual component.

4. Climate TRACE: the implementation contract

4.1 Version, endpoint, and provenance

The app targets **Climate TRACE API v7** (<https://api.climatetrace.org/v7/>). API v7 is an interface version, not a dataset release. The published [Climate TRACE data page](#), checked on 25 September 2026, listed **database release 5.11.0**, published 24 September, with monthly data through July 2026. The [v7 API reference](#) defines the request and response contract. Source API replies do not independently attest to their underlying release, so the translator reports the manually checked published release separately from `api_dataset_release`: `null`. Do not replace the null with an inferred version.

`config/climateTrace.js` builds URLs, requests `gas=co2e_100yr`, applies a 15-second deadline, and validates upstream responses with `shared/schemas/climateTrace.schema.js`. The backend is responsible for errors and cache policy. A failure or absent slug becomes unavailable; a real zero remains zero. The `USE MOCK_DATA=true` mode is explicit fixture mode and must not be presented as live Climate TRACE.

4.2 Aggregate dashboard product

Dashboard annual sector values use `GET /v7/sources/emissions`, with explicit year, `gas=co2e_100yr`, `gadmId`, and sector slug. For example:

```
GET https://api.climatetrace.org/v7/sources/emissions
    ?year=2025&gas=co2e_100yr&gadmId=UGA&sectors=transportation
```

`backend/services/climateTraceTimeseries.js` requests and caches each slug, year, and location; `backend/services/emissionsData.js` combines them into dashboard series and reconciliation. National history starts in 2015 and deliberately stops at the last complete annual year, currently 2025. District dashboard history starts in 2021. The translator exposes 2026 as partial context only; a partial year should not be compared to a complete year as though coverage were equivalent.

Upstream `emissionsQuantity` is **tonnes CO2e**. `shared/emissionsUnits.js` divides by 1,000,000 to show **MtCO2e**. Source calculations preserve precision; only display values are rounded. Negative land-sector removals must remain negative. Missing values remain null. For multi-slug UI sectors, the app returns a sector total only when each mapped slug for that year is present.

The nine displayed upstream slugs are grouped into six UI buckets: AFOLU uses `forestry-and-land-use`; Agriculture uses `agriculture`; Energy uses `power`, `buildings`, and `fossil-fuel-operations`; Transport uses `transportation`; IPPU uses `manufacturing` and `fluorinated-gases`; Waste uses `waste`. `mineral-extraction` is a tenth slug used in the country reconciliation but not shown as a sector card. This mapping lives in `config/ndcTargets.js`. Agriculture is a component of the NDC's wider AFOLU framing and is not an additional standalone 2022 pledge.

The reconciliation compares the ten-slug sum with `GET /v7/rankings/countries`. The response includes `country_total_mt`, `sector_sum_mt`, `ui_sector_sum_mt`, `delta_mt`, and `missing_slugs`. Missing components disable the relevant comparison. A delta can result from differences between upstream products, refresh timing, or rounding; preserve it and investigate rather than forcing equality. District all-sector totals use an unfiltered aggregate request, not the sum of visible source points or rounded cards.

4.3 Source and map product

`GET /v7/sources` is a different Climate TRACE product. `backend/services/climatetrace.js` uses it for top sources and the map, with year, gas, `gadmId`, `limit`, and `offset`. The map feed caps upstream rows at 3,000 and returns `truncated`; records without usable coordinates are excluded from the plotted layer. An administrative centroid may stand for emissions across a

larger area. The spatial-confidence endpoint explains located versus aggregated emissions. A map subtotal, a top-sources list, and a dashboard territorial aggregate have different coverage and must not be reconciled as identical inventories.

The source record may represent a directly located asset or an estimated administrative aggregation. The app carries source type and data-lineage metadata so users can inspect what a symbol means. A source record should never be described as a uniquely measured facility without checking its `is_asset` flag and other metadata.

4.4 District Translator product

The translator asks for Uganda source rows from `GET /v7/sources?`

`gadmId=UGA&gas=co2e_100yr&year=...`, paginating in blocks of 5,000 to a 50,000-row safety limit. The selected year's pages must be complete or analysis fails visibly. It detects repeated pages and conflicting duplicates and deduplicates by source/subsector key. Historical failures remain unavailable without blocking an otherwise valid selected year. The one-hour source-page cache and shared in-flight requests reduce repeat upstream calls. Retrieval timestamps refer to when source data was fetched, not when a boundary was clicked.

A selected `shapeID` identifies one of 135 pinned 2020 Uganda Bureau of Statistics district boundaries, distributed by UN OCHA/HDX and `geoBoundaries`. The client uses simplified display geometry; the server loads full analysis geometry. The shapes are not GADM IDs, and the 2020 boundaries do not claim to reflect every later administrative change. The polygon report includes entire emissions values of source centroids inside the selected shape. Missing-coordinate sources are omitted. Administrative-centroid values may describe a much larger region than the selected district. A zero-source result is a zero for matched points under this method, not proof of zero territorial emissions.

The dormant custom-polygon endpoint validates WGS84 geometry, closure, topology, holes and disjoint parts, a 512-position limit, a 300,000 km² area cap, and containment within the national boundary. It uses Turf for geodesic area. The Draw UI is hidden at present, but server geometry and tests remain. CSV and GeoJSON exports retain geometry, source records, year, units, provider, boundary provenance, filters, and coverage limitations. Translator analyses are not persisted in Postgres.

4.5 Cache, changes, and verification

`backend/services/climatetrace.js` caches a country snapshot for 24 hours and source/map results for one hour. `climateTraceTimeseries.js` caches successful slug/year/location values for one hour and failures for five minutes. Translator source pages cache for one hour. React Query adds browser-side freshness. Consequently, a response can be live-derived while still reflecting a recent cache entry; attach retrieval time when it matters.

When Climate TRACE publishes a release, check its data page and v7 reference, verify the response schema and Uganda values, decide whether the complete-year cap should move, run reconciliation, and only then change the translator's manually checked `TRACE_RELEASE` label. Do

not silently advance it with a calendar year. Run `npm run verify:climatetrace` for national slug/ranking checks and `node scripts/verify_translator.mjs` for independent source filtering. Record year, retrieval time, missing slugs, and any upstream failure in the verification report.

5. Targets, indicators, and derived outputs

Uganda's Updated NDC target configuration is bundled in `config/ndcTargets.js` and corresponding frontend data. It supplies sector targets, 2030 BAU references, conditionality, scope notes, and catalogue activities. Dashboard progress juxtaposes these official policy values with Climate TRACE estimates. Because the two sources have different methods, the gap is a decision signal that calls for interpretation, not an official verification of compliance. District geography is intentionally context-only because the target is national.

Indicator and evidence pages track variables beyond emissions and show provenance or QA flags. Ingested observations, if present, are another evidence stream and should not overwrite Climate TRACE history without an explicit source model. Data-lineage code in `frontend/src/lib/data-lineage.ts` connects chart sectors to upstream links, method notes, and refresh context. The Dashboard AI path builds a fact ledger and quotable facts, then `backend/routes/dashboardAi.js` and `dashboardAiCitations.js` constrain answers to available evidence and citations. The configured default model in `backend/services/openaiChat.js` is `gpt-4o-mini`; provider calls require `OPENAI_API_KEY` on the server.

Forecasts and finance screens are deliberately indicative. The 2030 prediction bundle extrapolates observed sector series with uncertainty; it does not become measured 2030 emissions. Finance costs and abatement are screening inputs from the catalogue, not verified transaction prices. The Policy Impact engine uses rule-based KCI case matching: sector 40%, intervention type 35%, region affinity 15%, and scale 10%, then aggregates the top three cases and exposes trade-offs, confidence, and case IDs. Its case schema is in `shared/schemas/policyImpact.schema.js`; the source cases live in `data/policy-cases/`.

6. Documents and AI workflow

The policy library is built from a Climate Policy Radar export rather than a live CPR API. `npm run build:documents` produces `data/policy/documents.json`; `npm run build:passages` produces searchable passage data; `npm run build:mcf` produces the MCF project corpus. `backend/routes/documents.js` serves library search, topics, passage search, project search, and project detail. The UI links to CPR or hosted PDFs and offers a policy pathway diagram. That urban transport pathway is an illustrative theory of change, not a measured attribution of tonnes to each intervention.

In `/documents/view`, selecting an item opens its metadata and the AI Policy Assistant. `backend/routes/policyAi.js` resolves the catalog URL, fetches and parses the PDF, selects text for the model, and returns structured analysis with page citations. Users can ask a question grounded in that PDF or use summary shortcuts. The model can still make mistakes; the page citation and source document remain the evidence to inspect. The server-side `OPENAI_API_KEY` must not be placed in a `VITE_` variable or shipped to the browser.

7. Protected writes and persistence

Site login and operator authorization are separate. `LOGIN_AUTH_ENABLED=false` currently bypasses site-wide Supabase sign-in, redirects `/auth`, and hides account controls. The Supabase code remains for future use. The operator unlock route `/api/v1/auth/session` exchanges a server-side passphrase for an `HttpOnly` session cookie. Ingestion writes are checked on the backend; non-browser jobs can use `x-api-key`. `INGEST_API_KEY` stays on the server. The legacy `VITE_INGEST_API_KEY` must remain unset because Vite exposes `VITE_` values in browser bundles. Role switching on the frontend grants no write permission.

Quick scan profiles a submitted file before import. The mapped Data Pipeline validates selected fields and units and writes observations when `DATABASE_URL` is configured. `database/` contains the Drizzle schema, migrations, and seed logic; `api/index.js` bootstraps the database in production cold starts. `SEED_DB=true` seeds bundled target/observation data on an intentional first deploy. `USE_DB_FALLBACK=true` is a development/demo fallback, not production persistence. The browser's `/my-work` drafts use `localStorage` and should be treated as local user state, not a shared ministry database.

8. Local development and production operation

At the repository root, install dependencies and run `npm run dev`. This starts the Vite frontend and Express API with the project's normal development bootstrap. `npm run dev:servers` starts both servers without the live verification bootstrap. The frontend listens at `http://localhost:8080`; the API listens at `http://localhost:8787`, with Vite proxying `/api`. `npm run start:api` starts the API alone. `npm run build` creates the frontend production bundle; `npm run tsc --build --force` checks TypeScript project references.

Production is one Vercel project: `api/index.js` hosts Express as a serverless function and the built SPA is copied into `public/`. Push to the connected production branch or run `vercel --prod --yes` for the linked project. `USE MOCK DATA=false` is necessary for live Climate TRACE. `FRONTEND_ORIGIN` sets the allowed production origin. `DATABASE_URL` enables persistence. `INGEST_API_KEY` enables protected writes. `OPENAI_API_KEY` enables AI features. No `VITE_API_BASE_URL` is needed for the normal same-origin deployment. `VITE_SUPABASE_*` values are dormant while site login is off; if login is restored, review their client exposure and configuration explicitly.

The health endpoints are operational checkpoints: `/api/health` identifies the API and live/mock mode; `/api/v1/health/full` reports fuller status, including persistence mode. Check them before interpreting a blank chart as zero emissions. Vercel upload limits are smaller than local Express limits (4 MB and four files versus 20 MB and ten files), and serverless execution has a 60-second limit in `vercel.json`. Long scans and very large datasets may need a dedicated API host.

9. Tests, release checks, and known limits

The main automated checks are `npm test` (Vitest), `npx tsc --build --force`, `npm run build`, and `npm run lint`. Focused translator and emissions tests cover schema handling, units, missing values, deduplication, pagination failure, geometry, and exports. `npm run test:e2e` and `npm run test:ux` are browser suites. `npm run scan:secrets` checks built output for private data. The live verification scripts depend on network access and should be run against a known year with their timestamps recorded; a passing unit test does not prove the upstream API is currently available.

Known interpretation limits are central to correct use:

- Climate TRACE is an independent model-based emissions source. It is not Uganda's official inventory or a district target allocation.
- Aggregate `/sources/emissions` values and located `/sources` records have different spatial coverage. They should not be forced to sum.
- The map may truncate at 3,000 rows; the translator has a 50,000-row safety cap and a source pagination caveat if future coverage exceeds the current one-page Uganda result.
- A 2020 UBOS district shape differs from a GADM dashboard district. Neither administrative vintage nor identifier is interchangeable.
- 2026 is a partial data year in the translator. The dashboard remains capped at the last complete year, 2025.
- An unavailable slug or historical year must remain unavailable. Zero means an observed numerical zero only when the upstream value supports it.
- Policy Impact, AI 2030, climate finance, risk, and pathway outputs are screening or illustrative products. Their provenance should remain visible in briefings.
- Re-enabling Supabase login or the Draw UI requires end-to-end authorization and spatial regression review; the retained code alone does not make either currently supported in the UI.

10. Maintenance map and external references

For a navigation change, update `frontend/src/App.tsx`, `frontend/src/lib/navigation.ts`, the in-app guide data, and this guide. For a Climate TRACE change, begin at `config/climateTrace.js`, `config/ndcTargets.js`, the relevant service and shared schema, then update tests, provenance copy, and the release note in this guide. For a boundary change, verify the source and license, rebuild both simplified display and full server geometry, and rerun translator checks. For a protected write, enforce permission in the route regardless of frontend role visibility. For a new evidence corpus, keep a reproducible build script and identify whether data is live, bundled, user-entered, or indicative.

Primary references

- Climate TRACE v7 API reference: <https://api.climatetrace.org/v7/docs/index.html>
- Climate TRACE data releases and data caveats: <https://climatetrace.org/data>

- Uganda updated NDC: <https://unfccc.int/NDCREG> (Uganda's 2022 submission; target values are bundled in repository configuration)
- Uganda district boundary provenance: <https://www.geoboundaries.org/api/current/gbHumanitarian/UGA/ADM2/> and <https://data.humdata.org/dataset/cod-ab-uga>
- Repository implementation entry points: `README.md`, `frontend/src/App.tsx`, `backend/server/createApp.js`, `backend/routes/emissions.js`, `config/climateTrace.js`, and `config/ndcTargets.js`

Glossary: *BAU* = business as usual; *CO2e* = carbon dioxide equivalent; *MtCO2e* = million tonnes CO2e; *GADM* = Global Administrative Areas identifier; *UBOS* = Uganda Bureau of Statistics; *MRV* = measurement, reporting, and verification; *CPR* = Climate Policy Radar; *MCF* = multilateral climate fund corpus; *KCI* = UNFCCC Katowice Committee of Experts; *TEF* = Transition Element Framework. A *located source* has usable coordinates; an *aggregate* is an emissions total for a specified sector and geography; a *partial year* has incomplete temporal coverage.

11. API quick map and troubleshooting

The following local endpoints are the stable places to start a trace through the code. All paths below are under the same-origin `/api/v1` prefix. The list is selective; `backend/server/createApp.js` and each route module remain the contract for exact parameters and response schemas.

- GET `/emissions/dashboard`, `/emissions/timeseries`, and `/emissions/progress`: the dashboard summary, annual history, and target comparison. Start with `backend/routes/emissions.js`, `emissionsData.js`, `climateTraceTimeseries.js`, and `config/ndcTargets.js`.
- GET `/emissions/map`, `/emissions/sources`, and `/emissions/spatial-confidence`: located points, top sources, and their spatial-coverage caveat. Start with `backend/services/climatetrace.js`.
- GET `/emissions/translator/metadata`, `/emissions/translator/districts`, and `/emissions/translator/sources`: boundary/provider metadata, selectable districts, and the year-specific source layer. Start with `backend/services/translator/`.
- POST `/emissions/polygon-insights`: the selected district or polygon report. Start with `backend/services/polygonInsights.js`.
- GET `/emissions/predictions`: indicative 2030 sector forecasts. Start with `backend/routes/emissions.js` and its prediction service.
- GET `/documents/passages/search` and `/documents/mcf/search`: CPR and MCF evidence discovery. Start with `backend/routes/documents.js`.
- POST `/dashboard/analyze` and `/policy-impact/forecast`: cited NDC AI and rule-based KCI/TEF forecasts. Start with their corresponding route modules.
- GET, POST, and DELETE `/auth/session`: operator status, unlock, and lock. Start with `backend/routes/authSession.js`.

- `POST /ingest/confirm`: protected mapped observation write. Start with `backend/routes/ingest.js`.

Trace a questionable dashboard number. Record the selected sector, geography, year, URL, retrieval time, and whether mock mode is active. Read `/emissions/dashboard` and `/emissions/timeseries`; inspect missing-slug and reconciliation fields; then reproduce the relevant `/v7/sources/emissions` call for each mapped slug. Convert tonnes to Mt only once. Compare the ten-slug sum with the country ranking for a national year and report the actual delta. Do not substitute map-point sums. If a district number is in question, check its GADM mapping and the unfiltered district aggregate call.

Trace a questionable translator number. Record boundary `shapeID`, analysis year, filters, and the provenance object. Ensure the request used the full server geometry and that all source pages were fetched. Check the centroid inclusion rule, source/subsector deduplication, source kind, missing-coordinate count, and selected year's completion. Compare to the raw `/v7/sources` rows with an independent filter. Do not compare the result directly with a GADM district dashboard total.

Trace an empty or stale screen. First check `/api/health` and `/api/v1/health/full`, then browser network responses, API logs, upstream timeout or schema errors, and cache age. An empty visible map can result from unlocated points, filters, truncation, or upstream availability. A denied write should be traced through the operator session and route authorization, not the UI role label. A PDF analysis failure should be traced through document URL resolution, fetch/parse status, and server AI configuration.

12. Qlik handoff extract

`data/exports/climate-trace-uganda-sources-2021-2025.csv` is a ready-to-import, UTF-8 CSV with a header row and 19,320 Uganda Climate TRACE v7 source/subsector records: 3,864 records in each complete year from 2021 through 2025. It was downloaded on 25 September 2026 from `/v7/sources` with `gadmId=UGA` and `gas=co2e_100yr`. Each year's returned page held fewer than the requested 5,000 records; duplicate source/subsector/year keys were rejected. The file preserves the API's source metadata, activity and capacity fields, and emission estimates, and flattens centroid coordinates into latitude, longitude, and SRID. `emissions_tco2e` is the source value in tonnes; `emissions_mtco2e` is that value divided by 1,000,000. `extracted_at_utc`, `api_version`, and `api_endpoint` document the extraction. The API replies do not identify their database release, so the CSV does not claim an attested release.

For Qlik, import the first row as field names, preserve `source_id` as an identifier, parse `year` as a whole number, parse coordinates and emissions as numeric, and retain negative emissions (removals). The 19,320 rows mix 230 point-source rows and 19,090 GADM aggregation rows across all five years. These are source records, not a national inventory table. Administrative centroids and sums by point location are not territorial district totals. The dashboard's aggregate `/sources/emissions` product remains the appropriate source for national or GADM-area total comparisons. Treat a blank activity, factor, or capacity as unavailable for that source; do not coerce it to zero.